

Point Cloud Registration: Multi-scale Approach Documentation

Table of Contents

1. [Introduction](#)
2. [Algorithm Overview](#)
3. [Installation and Requirements](#)
4. [Pipeline Stages](#)
 - [Stage 1: Coarse Registration](#)
 - [Stage 2: Medium Registration](#)
 - [Stage 3: Fine Registration](#)
 - [Stage 4: Ultra-Fine Registration](#)
5. [Key Functions](#)
 - [Point Cloud Loading](#)
 - [Preprocessing](#)
 - [Global Registration](#)
 - [ICP Refinement](#)
 - [Visualization](#)
6. [Parameters and Their Impact](#)
 - [Voxel Sizes](#)
 - [Maximum Point Limits](#)
 - [Distance Thresholds](#)
 - [Fitness Thresholds](#)
7. [Performance Considerations](#)
8. [Interpreting Results](#)
9. [Troubleshooting](#)
10. [Exporting Documentation as PDF](#)

Introduction

This documentation provides a comprehensive guide to the point cloud registration algorithm implemented in `Registered_Map.ipynb`. The algorithm performs registration (alignment) between two labeled point cloud maps:

1. A SLAM-generated labeled map in PLY format
2. A reference labeled map in PCD format

The algorithm uses a multi-scale approach to achieve high-quality alignment while maintaining reasonable performance. This documentation explains each step of the algorithm, the parameters used, and how to interpret the results.

Algorithm Overview

The point cloud registration algorithm follows a multi-scale approach with four main stages:

1. **Coarse Registration:** Initial alignment using global registration techniques with heavily downsampled point clouds
2. **Medium Registration:** Refinement of the initial alignment using moderately downsampled point clouds
3. **Fine Registration:** Further refinement using finely downsampled point clouds
4. **Ultra-Fine Registration:** Final refinement using very finely downsampled point clouds

Each stage builds upon the results of the previous stage, gradually improving the alignment quality. The algorithm also includes overlap detection, performance optimizations, and visualization tools to assess the quality of the registration.

Installation and Requirements

Dependencies

- open3d~=0.19.0
- numpy~=1.26.4
- ipython~=8.27.0

Installation

```
pip install -r requirements.txt
```

Pipeline Stages

Stage 1: Coarse Registration

Purpose: Obtain an initial rough alignment between the source and target point clouds.

Process:

1. Downsample the point clouds with a large voxel size (0.8)
2. Limit the maximum number of points to 75,000 for performance
3. Compute FPFH features for both point clouds

4. Perform global registration using Fast Global Registration (FGR)
5. If FGR doesn't provide good results, fall back to RANSAC-based registration

Parameters:

- Voxel size: 0.8 (balanced between performance and quality)
- Maximum points: 75,000
- Distance threshold: $2.5 \times \text{voxel size}$
- RANSAC iterations: 4,000 (reduced from 50,000 for performance)

Output:

- Initial transformation matrix
- Fitness score (higher is better)
- Inlier RMSE (lower is better)

Stage 2: Medium Registration

Purpose: Refine the initial alignment with more detail.

Process:

1. Skip this stage if coarse registration fitness > 0.9
2. Downsample the point clouds with a medium voxel size (0.2)
3. Limit the maximum number of points to 150,000 for performance
4. Apply the coarse transformation to the source point cloud
5. Perform ICP registration with point-to-point error metric

Parameters:

- Voxel size: 0.2
- Maximum points: 150,000
- Distance threshold: $2.0 \times \text{voxel size}$
- Maximum iterations: 30

Output:

- Refined transformation matrix
- Updated fitness score
- Updated inlier RMSE

Stage 3: Fine Registration

Purpose: Further refine the alignment with fine details.

Process:

1. Skip this stage if medium registration fitness > 0.9
2. Downsample the point clouds with a small voxel size (0.05)
3. Limit the maximum number of points to 400,000 for performance
4. Apply the medium transformation to the source point cloud
5. Perform two-stage ICP:
 - Stage 1: Point-to-point ICP with larger threshold ($2.0 \times \text{voxel size}$)
 - Stage 2: Point-to-plane ICP with smaller threshold ($1.0 \times \text{voxel size}$)

Parameters:

- Voxel size: 0.05
- Maximum points: 400,000
- Stage 1 distance threshold: $2.0 \times \text{voxel size}$
- Stage 2 distance threshold: $1.0 \times \text{voxel size}$
- Stage 1 maximum iterations: 30
- Stage 2 maximum iterations: 50

Output:

- Further refined transformation matrix
- Updated fitness score
- Updated inlier RMSE

Stage 4: Ultra-Fine Registration

Purpose: Achieve perfect alignment with very fine details.

Process:

1. Downsample the point clouds with a very small voxel size (0.03)
2. Limit the maximum number of points to 200,000 for performance
3. Apply the fine transformation to the source point cloud
4. Perform two-stage ICP:
 - Stage 1: Point-to-point ICP with larger threshold ($2.0 \times \text{voxel size}$)
 - Stage 2: Point-to-plane ICP with smaller threshold ($1.0 \times \text{voxel size}$)

Parameters:

- Voxel size: 0.03
- Maximum points: 200,000
- Stage 1 distance threshold: $2.0 \times \text{voxel size}$

- Stage 2 distance threshold: $1.0 \times \text{voxel size}$
- Stage 1 maximum iterations: 30
- Stage 2 maximum iterations: 50

Output:

- Final transformation matrix
- Final fitness score
- Final inlier RMSE

Key Functions

Point Cloud Loading

Function: `load_point_clouds()`

Purpose: Load the source and target point clouds from the specified files.

Process:

1. Load the SLAM-generated labeled map (source) from "maps/map from SLAM_labeled.ply"
2. Load the reference labeled map (target) from "maps/reference-labeled_map.pcd"
3. Print the number of points in each point cloud

Parameters: None

Output: Tuple containing the source and target point clouds

Preprocessing

Function: `preprocess_point_cloud(pcd, voxel_size, max_points)`

Purpose: Prepare point clouds for registration by downsampling, computing normals, and extracting features.

Process:

1. Downsample the point cloud using voxel grid downsampling
2. Further downsample if the point count exceeds the maximum limit
3. Optionally remove statistical outliers for finer detail levels
4. Compute normals with optimized parameters
5. Compute FPFH features for matching

Parameters:

- `pcd` : The input point cloud
- `voxel_size` : The voxel size for downsampling (smaller values preserve more detail)
- `max_points` : Maximum number of points to process for performance

Output: Tuple containing the downsampled point cloud and FPFH features

Global Registration

Function: `execute_global_registration(source_down, target_down, source_fpfh, target_fpfh, voxel_size)`

Purpose: Find an initial alignment between the point clouds.

Process:

1. Try Fast Global Registration (FGR) for quick initial alignment
2. If FGR gives good results (fitness > 0.3), use it directly
3. Otherwise, try RANSAC-based registration with simplified parameters
4. Apply initial centering transformation for better RANSAC performance
5. Use the better of FGR or RANSAC results

Parameters:

- `source_down` : Downsampled source point cloud
- `target_down` : Downsampled target point cloud
- `source_fpfh` : FPFH features of source point cloud
- `target_fpfh` : FPFH features of target point cloud
- `voxel_size` : The voxel size used for downsampling

Output: Registration result containing the transformation matrix, fitness score, and inlier RMSE

ICP Refinement

Function: `refine_registration(source, target, result_ransac, voxel_size, max_points)`

Purpose: Refine the initial alignment using Iterative Closest Point (ICP) algorithm.

Process:

1. Downsample the point clouds for faster ICP
2. Further downsample if the point count exceeds the maximum limit
3. Compute normals for point-to-plane ICP
4. Perform two-stage ICP:
 - Stage 1: Fast point-to-point alignment with larger threshold
 - Stage 2: Fine point-to-plane alignment with smaller threshold

5. Skip Stage 2 if Stage 1 achieves good fitness (> 0.8)

Parameters:

- `source` : Source point cloud
- `target` : Target point cloud
- `result_ransac` : Initial registration result from global registration
- `voxel_size` : The voxel size used for downsampling
- `max_points` : Maximum number of points to process for performance

Output: Refined registration result

Visualization

Function: `visualize_registration_result(source, target, transformation)`

Purpose: Visualize the registration result with multiple viewing options.

Process:

1. Apply the transformation to the source point cloud
2. Create three visualization options:
 - Standard colored point clouds (red for source, green for target)
 - Checkerboard pattern for better alignment assessment
 - Cross-section view for detailed alignment inspection
3. Save the final visualization as an image (`final_transformation.png`)

Parameters:

- `source` : Source point cloud
- `target` : Target point cloud
- `transformation` : The transformation matrix from registration

Output: Visual representation of the registration result

Parameters and Their Impact

Voxel Sizes

Voxel size is a critical parameter that controls the trade-off between detail preservation and computational efficiency.

Stage	Voxel Size	Impact
Coarse	0.8	Heavily downsamples the point clouds for fast initial alignment. Larger values improve performance but may reduce accuracy.
Medium	0.2	Moderately downsamples the point clouds for better alignment. Balanced between performance and accuracy.
Fine	0.05	Preserves more detail for accurate alignment. Smaller values improve accuracy but increase computation time.
Ultra-Fine	0.03	Preserves very fine detail for perfect alignment. Very small values can lead to overfitting to noise.

Maximum Point Limits

Maximum point limits prevent excessive computation time for large point clouds.

Stage	Max Points	Impact
Coarse	75,000	Limits the number of points for fast initial alignment. Larger values may improve accuracy but increase computation time.
Medium	150,000	Allows more points for better alignment. Balanced between performance and accuracy.
Fine	400,000	Allows many points for accurate alignment. Larger values improve accuracy but increase computation time.
Ultra-Fine	200,000	Limits points to prevent excessive computation while maintaining accuracy.

Distance Thresholds

Distance thresholds determine which point pairs are considered correspondences during registration.

Stage	Distance Threshold	Impact
Coarse	$2.5 \times \text{voxel_size}$	Larger thresholds allow for more correspondences, which is useful for initial alignment with large misalignments.
Medium	$2.0 \times \text{voxel_size}$	Moderate thresholds balance between allowing enough correspondences and preventing incorrect matches.

Stage	Distance Threshold	Impact
Fine (Stage 1)	$2.0 \times \text{voxel_size}$	Larger threshold for initial point-to-point alignment to allow more correspondences.
Fine (Stage 2)	$1.0 \times \text{voxel_size}$	Smaller threshold for refined point-to-plane alignment to focus on accurate matches.
Ultra-Fine (Stage 1)	$2.0 \times \text{voxel_size}$	Larger threshold for initial point-to-point alignment to allow more correspondences.
Ultra-Fine (Stage 2)	$1.0 \times \text{voxel_size}$	Smaller threshold for refined point-to-plane alignment to focus on accurate matches.

Fitness Thresholds

Fitness thresholds determine when to skip registration stages based on the quality of the previous stage.

Threshold	Value	Impact
Skip Medium	0.9	Skip medium registration if coarse registration fitness > 0.9. Higher values ensure medium registration runs in most cases.
Skip Fine	0.9	Skip fine registration if medium registration fitness > 0.9. Higher values ensure fine registration runs in most cases.
Skip Stage 2 in ICP	0.8	Skip Stage 2 of ICP if Stage 1 fitness > 0.8. Higher values ensure Stage 2 runs in most cases.
Use FGR directly	0.3	Use FGR result directly if fitness > 0.3. Lower values ensure RANSAC is used in most cases.

Performance Considerations

The algorithm includes several optimizations to balance performance and accuracy:

- Multi-scale approach:** Start with coarse alignment and progressively refine, which is more efficient than directly attempting fine alignment.
- Adaptive downsampling:** Further downsample point clouds if they exceed maximum point limits.
- Early stopping:** Skip registration stages if previous stages achieve good results.
- Two-stage ICP:** Use faster point-to-point ICP for initial alignment, then more accurate point-to-plane ICP for refinement.
- Optimized normal estimation:** Use appropriate parameters for normal estimation based on voxel size.

6. **Simplified feature matching:** Use Fast Global Registration when possible, which is faster than RANSAC.

Interpreting Results

The algorithm provides two main metrics to assess registration quality:

1. **Fitness score:** The ratio of inlier correspondences to the number of points in the source point cloud. Higher values indicate better registration.
 - o 0.0-0.3: Poor alignment
 - o 0.3-0.7: Moderate alignment
 - o 0.7-0.9: Good alignment
 - o 0.9-1.0: Excellent alignment
2. **Inlier RMSE:** The root-mean-square error of inlier correspondences. Lower values indicate better registration.
 - o 0.1: Poor alignment
 - o 0.05-0.1: Moderate alignment
 - o 0.01-0.05: Good alignment
 - o < 0.01: Excellent alignment

The algorithm also saves the registered point cloud at different stages for comparison:

- `registered_map_coarse.ply` : Result after coarse registration
- `registered_map_medium.ply` : Result after medium registration
- `registered_map_fine.ply` : Result after fine registration
- `registered_map_multiscale.ply` : Final result after all registration stages

Troubleshooting

If registration fails or produces poor results, consider the following:

1. **Insufficient overlap:** Ensure the point clouds have sufficient overlap. The algorithm includes overlap detection and will attempt to align centers if no overlap is detected.
2. **Insufficient features:** Point clouds with few distinctive features may be difficult to align. Consider adding artificial landmarks or using a different registration approach.
3. **Parameter tuning:** Adjust the voxel sizes, maximum point limits, and distance thresholds based on the characteristics of your point clouds.
4. **Initial alignment:** If you know the approximate alignment, consider providing an initial transformation matrix.

5. Noise and outliers: Clean the point clouds by removing outliers before registration.